

Fish Gill Health Classification Using Machine Learning

1. Project Overview

Problem Statement

Gill diseases are a major cause of mortality and economic loss in aquaculture. Manual inspection of fish gills is time-consuming, subjective, and requires expert knowledge.

This project aims to automatically classify fish gill images into:

- Healthy
- Unhealthy

using image processing and machine learning techniques.

Objectives

- Develop an automated fish gill health classification system.
 - Compare multiple feature extraction methods.
 - Evaluate multiple machine learning algorithms.
 - Select the best-performing model for deployment.
 - Build a user-friendly prediction interface using Streamlit.
-

2. Dataset Description

Classes

Class	Label
Healthy Gill	1
Unhealthy Gill	0

Dataset Distribution

Dataset Split	Healthy	Unhealthy	Total
---------------	---------	-----------	-------

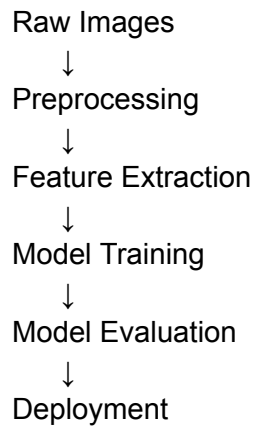
Train	86	170	256
Validation	22	42	64
Test	27	53	80
Total	135	265	400

Image Characteristics

- RGB images
 - PNG format
 - Variable image sizes
 - Controlled imaging conditions
-

3. Methodology

Workflow



4. Image Preprocessing

To ensure consistency across all images, a preprocessing pipeline was implemented.

Steps

1. Image Validation

Images were checked for:

- File existence
- Correct RGB channels
- Valid dimensions

2. Noise Reduction

Bilateral Filtering:

- Reduces noise
- Preserves important edges

3. Contrast Enhancement

CLAHE (Contrast Limited Adaptive Histogram Equalization):

- Enhances local contrast
- Improves visibility of gill structures

4. Image Resizing

All images resized to:

360 × 360 × 3

5. Normalization

Min-Max normalization:

Pixel Range:

0 – 255

↓

0 – 1

5. Feature Engineering

Several feature extraction techniques were investigated.

1. Raw Pixels

Flattened image representation.

Dimension:

$360 \times 360 \times 3 = 388,800$ features

2. Color Statistics

Extracted:

- Mean
- Standard deviation
- Skewness
- Kurtosis

for each color channel.

3. Color Histograms

RGB intensity distributions.

4. HSV Histograms

Color information in HSV color space.

5. Local Binary Patterns (LBP)

Texture descriptor capturing local texture variations.

6. Edge Features

Edge density and gradient information.

7. Histogram of Oriented Gradients (HOG)

Shape and structural descriptors.

8. Gabor Features

Frequency and orientation-based texture analysis.

9. Hybrid Features

Combination of multiple feature types.

6. Machine Learning Models

The following classifiers were evaluated:

Linear Models

- Logistic Regression

Support Vector Machines

- Linear SVM
- RBF SVM

Tree-Based Models

- Decision Tree
- Random Forest
- Gradient Boosting

Instance-Based Models

- KNN (k=3)
- KNN (k=5)

Probabilistic Models

- Gaussian Naive Bayes
-

7. Data Splitting Strategy

Stratified splitting was used to preserve class distribution.

Split Ratios

Dataset	Percentage
Train	64%
Validation	16%
Test	20%

Random Seed:

random_state = 42

8. Evaluation Metrics

The following metrics were used:

Classification Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- Matthews Correlation Coefficient (MCC)

Probability Metrics

- ROC-AUC

Error Analysis

- Confusion Matrix
 - Confidence Analysis
 - Threshold Optimization
-

9. Experimental Results

Best Model

Parameter	Value
Model	Logistic Regression
Feature Set	Raw Pixels
Feature Dimension	388,800

Performance

Metric	Value
Validation Accuracy	98.4%
Test Accuracy	100.0%
Precision	1.000
Recall	1.000
F1 Score	1.000
ROC-AUC	1.000

10. Feature Set Performance

Average Test Accuracy:

Feature Set	Accuracy
Raw Pixels	99.8%
Color Statistics	98.1%
HOG	95.4%
Gabor	92.9%
HSV Histogram	91.1%
LBP	89.7%
Color Histogram	62.6%

11. Model Comparison

Average Accuracy Across Feature Sets:

Model	Accuracy
Random Forest	91.9%
KNN (k=3)	91.8%

SVM Linear	91.1%
SVM RBF	91.1%
Decision Tree	90.9%
Logistic Regression	90.9%
KNN (k=5)	90.0%
Gradient Boosting	88.4%
Naive Bayes	80.8%

12. Error Analysis

Confusion Matrix

Results on the test dataset:

Actual	Predicted Healthy	Predicted Unhealthy
Healthy	27	0
Unhealthy	0	53

Findings

- No false positives observed.
- No false negatives observed.
- Strong class separation in feature space.

Important Observation

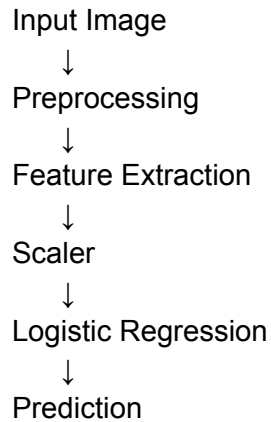
Perfect performance suggests one of the following:

- Dataset is highly separable.
- Imaging conditions are consistent.
- Potential dataset artifacts may exist.

External validation is recommended.

13. Deployment Strategy

Prediction Pipeline



Confidence-Based Review

Confidence	Action
> 0.7	Automatic Decision
0.4 – 0.7	Manual Review
< 0.4	Expert Verification

14. Streamlit Application

Features:

- Image upload
 - Real-time prediction
 - Confidence scores
 - Probability visualization
 - Deployment-ready interface
-

15. Project Structure

fish-gill-classification/
|

```
├── app/
│   ├── api.py
│   └── streamlit_app.py
├── src/
│   ├── config.py
│   ├── utils.py
│   ├── preprocessing.py
│   ├── prepare_data.py
│   ├── train_model.py
│   ├── fish_classifier.py
│   ├── predictor.py
│   └── evaluation.py
├── data/
├── models/
├── results/
├── requirements.txt
└── README.md
```

16. Limitations

- Limited dataset size (400 images).
 - Images collected under similar conditions.
 - External validation dataset unavailable.
 - Potential risk of dataset-specific artifacts.
-

17. Future Work

- Collect larger and more diverse datasets.
 - Test on multiple fish species.
 - Explore deep learning approaches (CNNs).
 - Implement explainability methods (Grad-CAM, SHAP).
 - Deploy as cloud API or mobile application.
-

18. Conclusion

A machine learning-based fish gill classification system was developed to distinguish healthy and unhealthy fish gills from images. Multiple feature extraction methods and machine learning algorithms were evaluated. Logistic Regression using raw pixel features achieved the highest performance, obtaining 100% test accuracy and 98.4% validation accuracy. The system demonstrates strong potential for assisting disease screening in aquaculture environments and provides a foundation for future intelligent fish health monitoring systems.